Treasure Row

Donovan Griffin, Chase Jones, Noah Higgs, Jonah Goodwine,
Cameron Ridgley, Ricki Davis, & Chi Martin



Functional Specifications

Table of Contents

1.0 Introduction 5
1.1 Goals and objectives 5
1.2 Statement of scope 5
1.3 Software context 5
1.4 Major constraints 6
2.0 Usage scenario 6
2.1 User profiles 7
2.2 Use-cases 7
2.3 Special usage considerations 8
3.0 Data Model and Description 8
3.1 Data Description 8
3.1.1 Data Objects 9
3.1.1.1 User 9
3.1.1.2 Storefront 9
3.1.1.3 Listing 10
3.1.1.4 Listing Image 11
3.1.1.5 Listing Size 11
3.1.1.6 Wishlist 11
3.1.1.7 Order 12
3.1.2 Relationships 13
3.1.3 Complete data model 14
3.1.4 Data dictionary 14
4.0 Functional Model and Description 17
4.1 Description for Function n 17

Registration and Login 17
Storefronts 18
Browse & Search 18
Listings 19
Wishlist 19
Cart & Checkout 20
Treasure Tokens 21
Access & Security 21
Returns 22
Admin 22
4.1.1 Processing narrative (PSPEC) for function n 23
4.1.2 Function n flow diagram 25
4.1.3 Function n interface description 25
4.1.4 Function n transforms 26
4.1.4.1 Transform 1 description (processing narrative, PSPEC) 26
4.1.4.2 Transform 1 interface description 26
4.1.4.3 Transform 1 lower level flow diagrams 27
4.1.4.4 Transform 2 description 27
4.1.4.5 Transform 2 interface description 27
4.1.4.6 Transform 2 lower-level flow diagrams 28
4.1.4.7 Transform 3 description 28
4.1.4.8 Transform 3 interface description 28
4.1.4.9 Transform 3 lower-level flow diagrams 29
4.1.5 Performance Issues 29
4.1.6 Design Constraints 29
4.2 Software Interface Description 30
4.2.1 External machine interfaces 30

4.2.2 External system interfaces 30
4.2.3 Human interface 30
4.3 Control flow description 30
4.4 Inherted Code 31
4.4.1 Code Contributions 31
4.4.2 Code Contributions 34
4.4.3 Code Contributions 36
4.4.4 Code Contributions 38
4.4.5 Code Contributions 38
4.4.5.1Coding Contributions - I4 40
4.4.5.2 Coding Contributions - I5 41
4.4.6 Code Contributions 44
5.0 Behavioral Model and Description 46
5.1 Description for software behavior 47
5.1.1 Events 47
5.1.2 States 47
5.2 State Transition Diagrams 48
5.3 Control specification (CSPEC) 48
6.0 Restrictions, Limitations, and Constraints 48
7.0 Validation Criteria 49
7.1 Classes of tests 50
7.2 Expected software response 51
7.3 Performance bounds 51
8.0 Revision Logs 51

1.0 Introduction

Our team, Treasure Row, is tasked with developing a cloud-hosted campus marketplace for students enrolled at Hampton University. The system will let users log in, browse listings, and post listings through a web-based interface. We chose to develop a fashion marketplace that will allow Hampton University's large fashion community to buy and sell original clothing.

1.1 Goals and objectives

Our goal for our product is to deploy a fully operational cloud-hosted campus marketplace intended for Hampton University students. The system will allow users to securely log in, browse listings, and post simple listings through a web-based interface. We will be using several AWS services to help us create, edit, and deploy our product.

1.2 Statement of scope

The software developed for this cloud-based marketplace will allow Hampton University students to buy and sell original clothing. The system will include user authentication, listing management, and database-backed storage through a web-based user interface.

- **Inputs:**
  - User login credentials
  - User-created listing information (title, description, photos, category, etc.)
  - User actions (deleting listings, filters for listings, edit/add listings, etc.)
- **Processing functionality:**
  - Authentication and authorization of users using secure password hashing
  - Creation, storage, retrieval, and updates of listings and purchases
  - Listing ownership and authorization, ensuring users can edit only the listing they created
  - Cloud-hosted database storage using AWS
- **Outputs:**
  - A catalog of active marketplace listings
  - User-owned listings with options to edit or delete

1.3 Software context

For any software being used for our product, the following behavioral requirements are needed:

- Security
  - The users' login and listing information must be secure at all times
- Scalability

    - The marketplace must be able to store a vast amount of users and user listings while the number for both is continuously growing
- Reliability
    - The marketplace must have little to no crashes and cannot be vulnerable to cyber threats
- Usability
    - Users must be able to access the marketplace across macOS, Windows, and multiple web browsers

1.4 Major constraints

The following constraints will affect product development:

- **Technical Constraints:**
    - **Cross-platform compatibility:** The system must function consistently on macOS, Windows, and across web browsers
    - **Listing ownership:** The system must be developed using AWS which requires compatibility with AWS hosting, networking, and database solutions.
    - **AWS Free tier:** Since the team only has access to AWS Free Tier, the system must be deployed using the free tier, limiting cloud resources and requiring efficient configuration
- **Resource Constraints:**
    - **Time Constraints:** The product has a set timeline that it must be completed within. If it is not completed within the given timeline, it will need to be handed off to the next class to be refined.

2.0 Usage scenario

Users of *The Vault* will consist of Hampton University students who will be able to access the platform through a browser and application. All users will gain access to the application by registering for an account using verified university credentials and a secure username and password. Once authenticated, users will be directed to our homepage, where they will be able to browse the catalog for available clothing items from verified student brands. Shoppers will also be able to view details for each item as well as filter through the items. Student brand owners will be able to create an account for their businesses and create new listings by entering all the information on the clothing. Users are able to edit and delete any listing that they created, while administrators have permissions to manage all listings. After a user has successfully finished any task of their choosing, they can log off of the system securely.

2.1 User profiles

The Vault is made to support multiple user categories. Each category consists of different roles, permissions, and goals.

Student Shopper
The student shopper is a Hampton University student who will use the platform to browse the clothing listings given by the student brand owner. The goal of this user is to discover and view any available items posted by the brand owners. Shoppers can view listings, search and filter items, and message sellers, but they cannot modify listings.

Student Brand Owner
A student brand owner is a Hampton University student who manages and owns a clothing brand and contributes their brand to *The Vault*. The user is able to create, edit, delete, and add to the browser listing. Brand owners are responsible for maintaining the information for their products and managing their listings.

Administrator
An administrator is a user given privileges and responsibilities to oversee the system. Administrators are given access to all listings and can modify them (i.e., edit, remove, etc.) if any listings violate platform guidelines. This role ensures the system's integrity, policy is enforced, and overall platform management.

2.2 Use-cases

The following use-cases represent the primary interactions users have with The Vault:

- User registers for an account using Hampton University credentials or similar
- User logs into the system using a secure username and password
- Student shopper browses and searches marketplace listings
- Student brand owner creates a new clothing listing
- Student brand owner edits an existing listing they own
- The student brand owner deletes a listing they own
- Administrator views, edits, or deletes any listing
- User logs out of the system securely

2.3 Special usage considerations

The following special use cases explain exceptional or restricted scenarios within The Vault's programming that ensure security, integrity of the platform, and enforcement of policy, procedures, and rules.

The system confirms listing ownership in the event of an unauthorized listing edit, which occurs when a student user tries to edit a listing they do not own. Access is refused and a suitable error message is shown if the user is not the owner. The listing is still in effect.

Administrators can change, remove, or edit any listings if they are found to violate any platform's rules. This ensures system compliance and preserves integrity by updating the platform's marketplace to reflect an accurate representation of the platform.

Lastly, authentication fails and an error message appears if a user enters incorrect login information. The user must try to log in again using legitimate credentials since they are still logged out.

3.0 Data Model and Description

This section describes the information domain for The Vault. There will be descriptions identifying the data objects the system manages/affects and the relationships among them. Additionally, the data model included, supports the core requirements for the system such as such as user authentication, storefront creation, listing management, ownership enforcement, inventory tracking, wishlist management, listing storage, checkout, order tracking, return handling, and administrative control.

All structured data is stored in a cloud-hosted relational database using AWS RDS (PostgreSQL)

3.1 Data Description

The Vault manages several categories of data:

- User account data
- Storefront and listing data
- Order and transaction data
- Cart data
- Return request data
- Wishlist data

More specifically, user account data supports secure authentication and role-based authorization.

Storefront and listing data support product creation, modification, and ownership enforcement within the marketplace. Order and cart data supports the checkout lifecycle and transaction tracking. Wishlist data supports user engagement by allowing shoppers to save items of interest. All data elements are stored within a structured relational database to maintain consistency and enforce relationships between entities.

3.1.1 Data Objects

The following data objects represent the primary entities required to support the system's functional requirements.

3.1.1.1 User

The User object represents a registered Hampton University student or administrator who accesses The Vault. Each user is assigned a role that determines their level of access and permissions within the system.

Users operate under a unified account model, meaning a signal account may function as both a shopper and a brand owner depending on whether a storefront has been created.

Attributes:

- user_id (unique identifier)
- username
- email
- password_hash (stored securely using hashing)
- role (shopper, brand_owner, admin)
- Created_at (timestamp of account creation)

3.1.1.2 Storefront

The Storefront object represents a brand profile created by a user. When a user creates a storefront, they gain seller capabilities within the system.

A storefront serves as the organizational entity under which listings are created and managed.

Attributes:

- storefront_id
- owner_id (references User)
- brand_name
- bio
- logo_url
- banner_url

- contact_info
- preview_image_1
- preview_image_2
- preview_image_3
- preview_image_4
- categories
- is_active
- created_at
- updated_at
-

3.1.1.3 Listing

The Listings object represents an item offered for sale by a brand owner. Each listing is associated with a specific storefront.

Listings may operate under two fulfillment models:

- **Inventory-based:** Limited quantity available
- **Made-to-order: (preorder):** Produced after purchase with no fixed inventory count

The system enforces inventory validation for inventory-based listings and automatically updates status when quantities reach zero.

Attributes:

- listing_id (unique identifier)
- storefront_id (references Storefront)
- title
- description
- price
- fulfillment_type
- quantity_on_hand
- sizes_available
- is_made_to_order
- status
- storefront_restore_status
- deleted_restore_status
- created_at
- updated_at

Inventory control is a significant factor and for inventory-based listings, quantity_available is displayed to shoppers when browsing or viewing a listing (e.g., sizes/quantity available). The system validates the quantity available when an item is added to cart and again at checkout.

For made-to-order listings, quantity limits do not apply, and orders can be placed without stock manipulation

3.1.1.4 Listing Image

The Listing image object represents image records associated with a listing.

Images are stored externally in Amazon S3, while the corresponding image URLs are stored in the database. A listing may contain multiple images, with one designated as the primary image.

Attributes:

- Image_id
- Listing_id (references Listing)
- Image_url
- Is_primary
- Created_at

3.1.1.5 Listing Size

The Listing Size object supports size-based inventory tracking for listings that require multiple size options.

Each size entry maintains a quantity value and is associated with a specific listing.

Attributes:

- Size_id
- Listing_id (references Listing)
- Size_label
- Quantity
- Created_at

3.1.1.6 Wishlist

The Wishlist object represents saved listings associated with a user account. It allows shoppers to store items for future review or purchase.

Attributes:

- Wishlist_id
- User_id (references user)
- Listing_id (references Listing)

- created_at

3.1.1.7 Order

The Order object represents a transaction initiated by a shopper during checkout. It tracks the lifecycle of a purchase from initiation through fulfillment.

Each order records both the buyer and the seller to support transaction history tracking and administrative oversight.

Attributes:

- order_id (unique identifier)
- user_id (references User)
- confirmation_number (unique, system-generated)
- buyer_first_name
- buyer_last_name
- buyer_email
- buyer_contact
- buyer_meeting_location
- total_amount
- order_status (SUBMITTED)
- created_at

3.1.1.8 Order Item

The Order Item object represents an individual line item within an order. Each order may contain multiple order items corresponding to the listings purchased during checkout.

Attributes:

- order_item_id (unique identifier)
- order_id (references Order)
- listing_id (references Listing)
- item_name
- quantity
- price_at_purchase
- selected_size

3.1.1.9 Cart Item

The Cart Item object represents a listing added to a user's shopping bag. Cart items are stored persistently in the database and loaded into the session on login. The cart is cleared upon successful checkout.

Attributes:

- cart_item_id (unique identifier)
- user_id (references User)
- item_id (references Listing)
- item_name
- price
- quantity
- size
- created_at

3.1.1.10 Return

The Return object represents a return request submitted by a user after a purchase. It tracks the reason, damage status, and optional damage image uploaded to S3.

Attributes:

- return_id (unique identifier)
- user_id (references User)
- order_number
- reason
- has_damage
- damage_image_url (S3 URL, optional)
- status (pending, reviewed, resolved)
- created_at
- updated_at
- is_deleted

3.1.2 Relationships

The following relationships among data objects exist as follows:

- A User may create one Storefront.
- A Storefront may have multiple Listings.
- A Listing may have multiple Listing Images.
- A Listing may have multiple Size records (for inventory-based items).
- A User may maintain multiple Cart Items in their persistent shopping bag.
- A User (shopper) may create multiple Orders.
- An Order may contain multiple Order Items, each referencing a specific Listing.
- An Order may have zero or one associated Return record.
- A User may submit multiple Return requests.
- A User may maintain multiple Wishlist entries.
- An Administrator may access and manage all Users, Storefronts, Listings, Orders, and Returns.

These relations enforce storefront and listing ownership, transaction traceability, structured inventory management, and admin control throughout the system.

3.1.3 Complete data model

The Vault's Entity-Relationship Diagram (ERD) showcases the data structure of the system. The diagram shows primary entities: User, Storefront, Listing, Order, Order Item, Cart Item, Wishlist, and Return, along with their major attributes, primary keys, and foreign key relationships.

The ERD demonstrates how:

- A User may create a Storefront.
- A Storefront may hold multiple Listings.
- Listings may include associated images and size-based inventory records.
- A User may maintain multiple Cart Items in their persistent shopping bag.
- Shoppers (Users) may place Orders for Listings.
- Each Order may contain multiple Order Items referencing specific Listings.
- Orders may optionally generate Return records.
- Users may save Listings to a Wishlist.
- Admins may access and manage all system entities.

The data model enforces relational integrity, ownership constraints, and transactional traceability through structured foreign key relationships. It supports authentication, listing lifecycle management, cart management, checkout, order tracking, return handling, and admin oversight within a centralized relational database environment.

3.1.4 Data dictionary

The following data dictionary defines the primary attributes used within The Vault system. It provides a description of each attribute for clarity of its role within the data model.

Attribute	Description	Attribute	Description
user_id	Unique identifier for each registered user	username	User created account name

email	User email address	password_hash	Securely hashed password created by user
role	Defines permission level (shopper, brand_owner, admin)	created_at	Timestamp of record creation
storefront_id	Unique identifier for a storefront	owner_id	Identifies the user who owns the storefront
contact_info	Storefront contact information	categories	Comma-delimited list of storefront style categories
preview_image_1-4	S3 URLs for storefront preview images	listing_id	Unique identifier for each listing
title	Listing title	description	Detailed listing information
price	Listing price	fulfillment_type	Defines whether listing is inventory based or made to order
quantity_on_hand	Current stock level	sizes_available	Available sizes for the listing
is_made_to_order	Indicates if listing is produced after purchase	status	Listing state (active, sold out, inactive, deleted)

storefront_restore_status	Remembers listing state before storefront deactivation	deleted_restore_status	Remembers listing state before soft delete
image_id	Unique identifier for listing image	image_url	Reference to image stored in Amazon S3
is_primary	Indicates primary image for listing	size_id	Unique identifier for size record
size_label	Size designation (e.g., S, M, L, XL)	quantity	Available quantity for a specific size
wishlist_id	Unique identifier for wishlist entry	order_id	Unique identifier for each order
confirmation_number	Unique system-generated order reference number	buyer_meeting_location	Location submitted by buyer for in-person exchange
buyer_contact	Buyer contact information	buyer_email	Buyer email address submitted at checkout
total_amount	Total transaction amount	order_status	Order lifecycle state (SUBMITTED)
order_item_id	Unique identifier for each order line item	price_at_purchase	Listing price recorded at time of purchase
selected_size	Size selected by buyer at checkout	cart_item_id	Unique identifier for each cart item
return_id	Unique identifier for return request	order_number	Order confirmation number referenced in return

has_damage	Indicates if returned item has damage	damage_image_url	S3 URL for optional damage image
return_status	Status of return (pending, reviewed, resolved)	is_deleted	Soft delete flag for return records

The data dictionary will be maintained electronically and updated as development progresses.

4.0 Functional Model and Description

This section describes the functional breakdown of the Vault system, including processing logic, interfaces, data flow, performance considerations, and constraints. The system follows a client-server architecture deployed on AWS using Elastic Beanstalk for application hosting and Amazon RDS (PostgreSQL) for database services.

4.1 Description for Function n

Registration and Login

- The user enters name, email, and password on the Login/Sign up page
- The server (auth_services.py) checks that the email ends in @my.hamptonu.edu or @hamptonu.edu
    - A non-Hampton email is rejected, and this is enforced not just in the browser, but on the server side
- The server checks for duplicate accounts
- The password is hashed with Werkzeug (pbkdf2: sha256), and the user record is stored with the title shopper by default
- If all the login information is valid, a Flask session is then created containing the user account and cart state, and the user is given access to the homepage
    - If invalid information is entered, the server will return a 401 try screen
- Logout exits the session

Storefronts

- A logged-in user can click the “Create Storefront” tab, which opens /storefronts/create
- The user has to enter a store banner image & logo, brand name, contact info (instagram/ email), store description, store categories (streetwear, vintage, athleisure, accessories, casual, earthy, retro, and other), and up to 4 preview images
    - Each uploaded image is validated (only if it's a PNG, JPG, JPEG, GIF, or WEBP file), given a filename, uploaded to S3, and has its URL saved in the storefront table
- The new storefront appears as a card on the homepage
    - The user will gain the benefits of a seller
    - A user may own one storefront
- “Enter Storefront” opens /storefronts/<storefront_id>
- “Edit Storefront” runs an update on the record, and only the owner of the storefront or an admin can do so
- Inactive storefronts are hidden from the public
    - They stay visible to the owner and admins
- “Deactivate Storefront” option sets is_active to false since it’s unfinished

Browse & Search

- After logging in, the shopper is directed to the storefront homepage, which loads all the active storefronts as cards (GET /api/storefronts)
- The top tabs are Storefronts, My Storefront, Sell an Item, My Bag, Returns, My Account, and About
- To the side the Menu has Shop(leads user to All Storefronts, My Bag, Returns), Sell (My Storefront, Create Storefront, Create Listing), and More(My Account, About, Sitemap, Log Out)
- Storefront cards show item count and categories with Previous-Next paging
- Create Listing allows up to 4 preview images to be showcased, size options of One Size, XS, S, M, L, XL, XXL(with quantity bar to set how much each size has) or a Made to Order option, and price
- Each card opens that storefront’s page (/storefronts/<storefront_id>), where all of its listings will be shown with the new set appearing first
    - Listings that are deleted or hidden aren't visible to shoppers
    - Inactive storefronts are hidden from shoppers

- A listing with a quantity of 0 will show a disabled (greyed out) Sold Out instead of an add to bag option
- Known issues:
    - The search only covers storefront names. A storefront's listing can't be searched, sorted, or filtered.
    - Categories, although set and stored on storefronts, can't be used to filter results
    - The homepage navigation tabs, All categories, New drops, Hot!, Explore brands, and About Sitemap, don't lead you anywhere

Listings

- The owner opens "Create Listing" and then picks one of their storefronts, enters the name of the item, adds a description, sets the price, uploads the image, and selects the size (up to 6 sizes)
    - owner can set a quantity per size or make it made-to-order
- The server validates required fields and rejects if there are negative inputs for price/ quantity
    - listing_sizes records this information
- Images uploaded to S3 are recorded in listing_images
- The listings show on the storefront
    - Once the quantity of a listing reaches 0, the add to bag button is replaced by a sold out tag
- The owner can edit listings
    - Listing statuses can toggle between hidden and active
    - Listings can be deleted, but it's not a definite delete
        - They are kept in deleted_restore
- Before a listing is edited or deleted, there are checks to make sure the user owns the listing
    - However, Admins can modify any listing

Wishlist

- A shopper can click the gold lock icon on each listing card
    - Clicking the lock icon again can remove it
- The listing will appear in the "Wishlist" tab in My Account
    - Displays each saved item with an add-to-cart and remove button
- The page loads the saved wishlist from the server (/api/wishlist)
    - The browser copy is a fallback

Cart & Checkout

- The shopper selects a size and adds the item to the shopper's bag
    - The server validates the quantity of the item on hand at the moment
    - The price and name always come from the database
    - A sold-out listing will display a disabled "Sold Out" label where the "Add to Cart" would be
- Items in the cart are saved to the database
    - Loaded into the session at login
    - Status survives logout
- On the My Bag page, the total is calculated, and the shopper can click "Checkout"
    - The shopper enters their full name, contact info/email, and a meeting location for an in-person exchange
        - Orders are paid with Treasure Tokens
            - No real money or card information is collected
            - 1 Treasure Token equals $1 of the price of the listing
    - Stock and prices are validated again
    - The server rejects an empty cart or a balance too low
- The server then runs these steps:
    - Create the order (status SUBMITTED)
    - Create the order items with price_at_purchase
    - Move the order total from the buyer's wallet into escrow
    - Decrement the inventory
    - Clear the cart
    - Remove the purchased items from the wishlist
- A confirmation number is then generated and shown on the order details page
    - The page shows the order date, order status, order total in Treasure Tokens, buyer information, meeting location, the items purchased, and the pickup code
    - The order can be looked up with that number
    - Only the buyer can see the pickup code
    - Viewing an order requires the logged-in user to be the buyer; otherwise, "Access denied" will pop up
- Order status:
    - Submitted at checkout
    - Completed once every seller's escrow is done
    - Cancelled when all escrows are refunded

Treasure Tokens

- Every year, a wallet opens automatically with a starting balance of 200 Treasure Tokens as a default
    - The default amount is set by TOKEN_STARTING_BALANCE
    - The balance is shown in the header balance, My Account, and the Wallet Tab with a purchase/sell history
- Escrow:
    - Tokens paid at checkout are held per seller until pickup is completed
    - Each escrow has a one-time pickup code given to the buyer (shown on the order confirmation page and Wallet tab) to give to the seller to complete the transaction
        - The code is never stored
        - The code is derived from the server's SECRET_KEY
- The release is at the meetup
    - The buyer gives the seller the pickup code after they've received their items
    - The seller enters the code on the My Account page on the Sales History tab
        - It can also be reached from “Pickups to confirm” on My Storefront
    - Only the seller of that escrow can release it
        - The tokens then move to the seller’s wallet and it will show as “Picked up”
    - After 5 wrong codes, the escrow is locked and an admin must review it
- Cancellation & Refund
    - Before pickup, the buyer, the seller, or admin can cancel
    - Once the refund is processed the tokens return to the buyer
    - After pickup, the tokens can no longer be refunded by users
    - Every balance is recorded in the ledger
        - The wallet tab shows the history(starting balance, escrow hold, release, and refund)
    - A balance can't go negative and checkout locks the buyer’s wallet

Access & Security

- Almost every page requires a signed-in user except for the landing page
- The user’s profile comes from the server session
- Prices always come from the database
- Forms are protected against cross-site requests
- Uploads are limited to real images under 5MB in certain folders

- Hampton e-mail check rejects lookalike domains

Returns

- On the returns page, the buyer reads the Return Instructions listed at the very top then enters a confirmation number (given once an order has been confirmed), indicates whether the item is damaged or not, has an option to upload an image of the damaged item, and a return reason
    - The instructions state:
        - Contact the seller first
            - Sellers most include contact information on their storefronts
        - If the seller can't be reached then email TreasurerowAdmins@my.hamptonu.edu
        - The form down below must be completed so the request can be tracked
    - The server verifies the confirmation number, reason for return, that the confirmation number belongs to one of the signed-in user's own orders, and if there were answers to the damage questions
        - If the confirmation number comes back invalid it will show "We couldn't find that order on your account. Check the confirmation number"
    - If the item is damaged, there's an option for the buyer to upload a photo
        - The upload will be checked to be a real image and if it meets the 5 MB limit
        - The photo will then be stored in S3
    - The return request is saved with the buyer's user_id and status pending
    - An admin reviews each request in the dashboard with the buyer's username and contact information
        - It will go through the process of being reviewed and then be dealt with
            - The status of the request can be set to pending, reviewed, or resolved
            - There is no required order the status value goes in; just validation of the status value
                - A request can jump from pending to request
    - Tokens can be refunded by
        - canceling before pickup
        - an admin refunding an escrow that is locked
    - A user submitting a return doesn't refund tokens

Admin

- The admin signs in on the Admin Access login page

    - The session store role will be admin
        - Admin credentials come from ADMIN_USERNAME and ADMIN_PASSWORD in .env
        - Every admin route runs require_admin(), which will read the role from the session
- The admin dashboard (/admin/dashboard) has all the users, listings, storefronts, and returns information (GET /admin/users,/listings,/storefronts,/returns)
- There are dashboard sections for wallets and escrows
    - It lists every user's balance and grants 200 welcome tokens
- Admin can hold a list and lock pickup orders
- Admin can issue refunds to orders
- Admin can delete users, listings, and storefronts (DELETE /admin/…)
    - When deleting a listing, the record is kept for admin tracking
        - There's an option to deactivate a listing; it can later be restored
- The admin updates a return's status to pending, reviewed, or resolved (PATCH /admin/returns/<id>)
    - The service has no priority order, so an admin can bounce from pending to resolved

4.1.1 Processing narrative (PSPEC) for function n

A processing narrative for function n is presented.

User registration:

When a user selects "Register," the system:

- Accepts input data (name, email, password)
- Validates email format and ensures it belongs to the Hampton University domain
- Checks for duplicate accounts in PostgreSQL (RDS)
- Encrypts the password using secure hashing
- Stores user information in the Users table
- Returns a confirmation or error message

User authentication(login/logout):

- User submits email and password.
- The system retrieves the stored password hash from RDS.
- Compares the input password with the hashed value.
- If valid, it generates a secure session token.
- Grants access to marketplace features.

Create listing:

- Authenticated user selects “Create Listing.”
- User inputs:
    - Item name
    - Description
    - Price
    - Category
    - Image upload
- System validates required fields
- Stores listing in Listings table in RDS
- Uploads image to AWS storage (S3). Displays listing in the marketplace feed

Edit/delete listing:

- User selects the listing they own
- System verifies ownership
- Allows modification or deletion
- Updates or removes listing from RDS

Browse & search listings

- User accesses homepage
- System retrieves active listings from RDS
- Displays listings sorted by most recent
- Search filters apply SQL query filters (price, category, keyword)

Admin management:

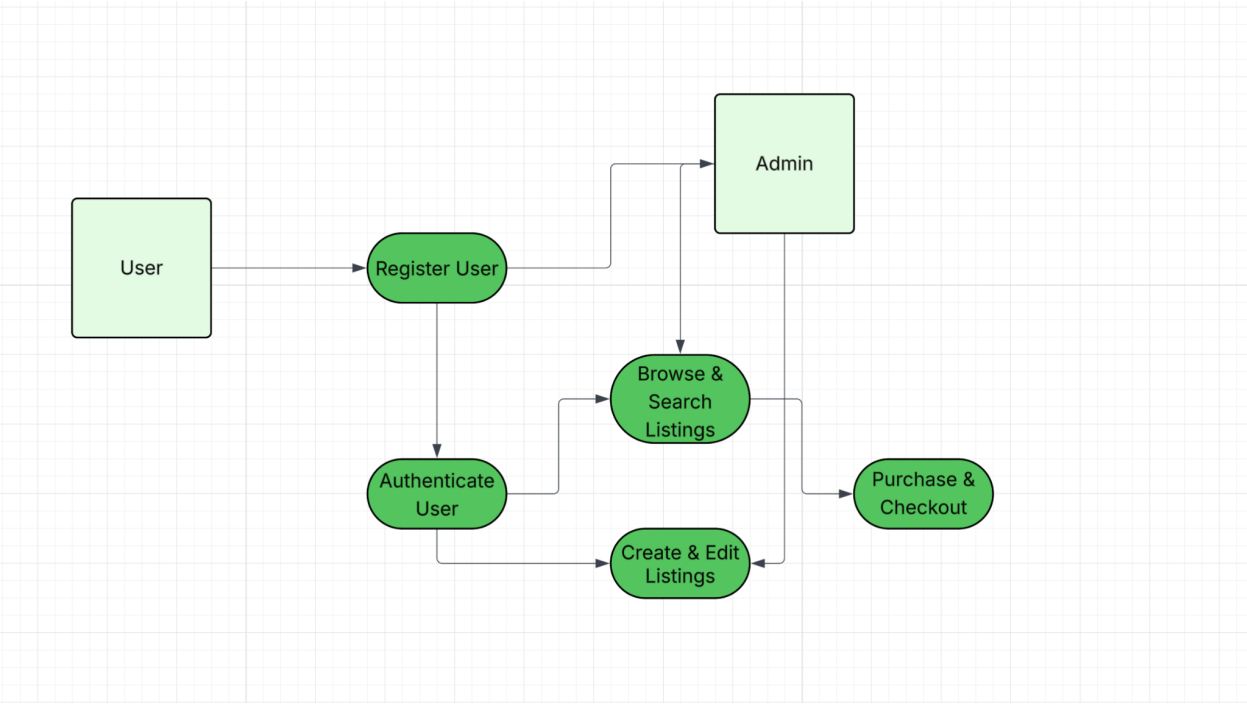
Admin users can:
- View all listings
- Remove inappropriate content
- Suspend user accounts

Purchase & checkout:

- User selects item

- Confirms purchase
- System verifies listing availability
- Marks item as sold in the database
- Notifies seller

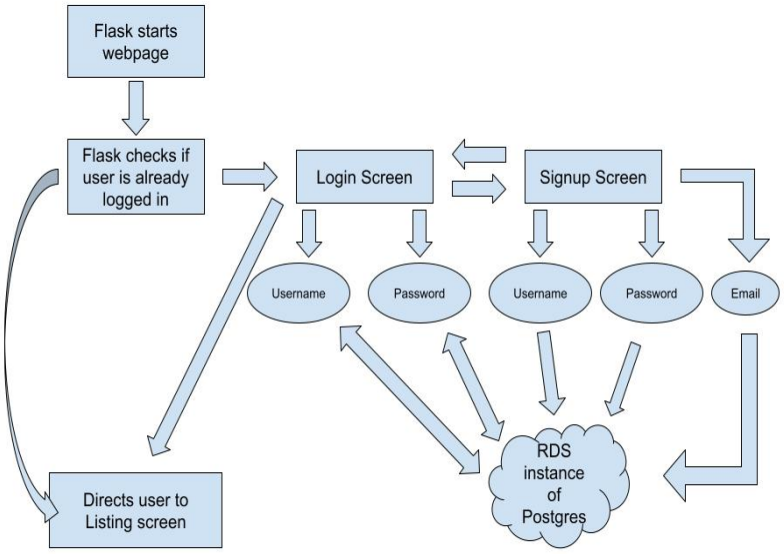
4.1.2 Function n flow diagram



4.1.3 Function n interface description

A detailed description of the input and output interfaces for the function is presented.

- Input interface: JSON is a list of keys and values. The input interface expects three pieces of input: email, username, and password. If the data type doesn't match the JSON rules, such as a phone number being entered instead of an email, the interface rejects it at the window before it reaches the database.
- Output interface: If the login is successful, a 200 code response is sent back along with routing to the listing screen. But, if the login fails, a 401 error will be sent.



4.1.4 Function n transforms

A detailed description for each transform (subfunction) for function n is presented. Section 4.1.4 is repeated for each of k transforms.

4.1.4.1 Transform 1 description (processing narrative, PSPEC)

This transform manages how students join and enter the marketplace.

During registration, the system accepts a name, email, and password. During this process, the system ensures the email belongs to the Hampton University domain (@hamptonu.edu), the username/email isn't a duplicate, and the password is hashed before storing it in the PostgreSQL database on AWS RDS.

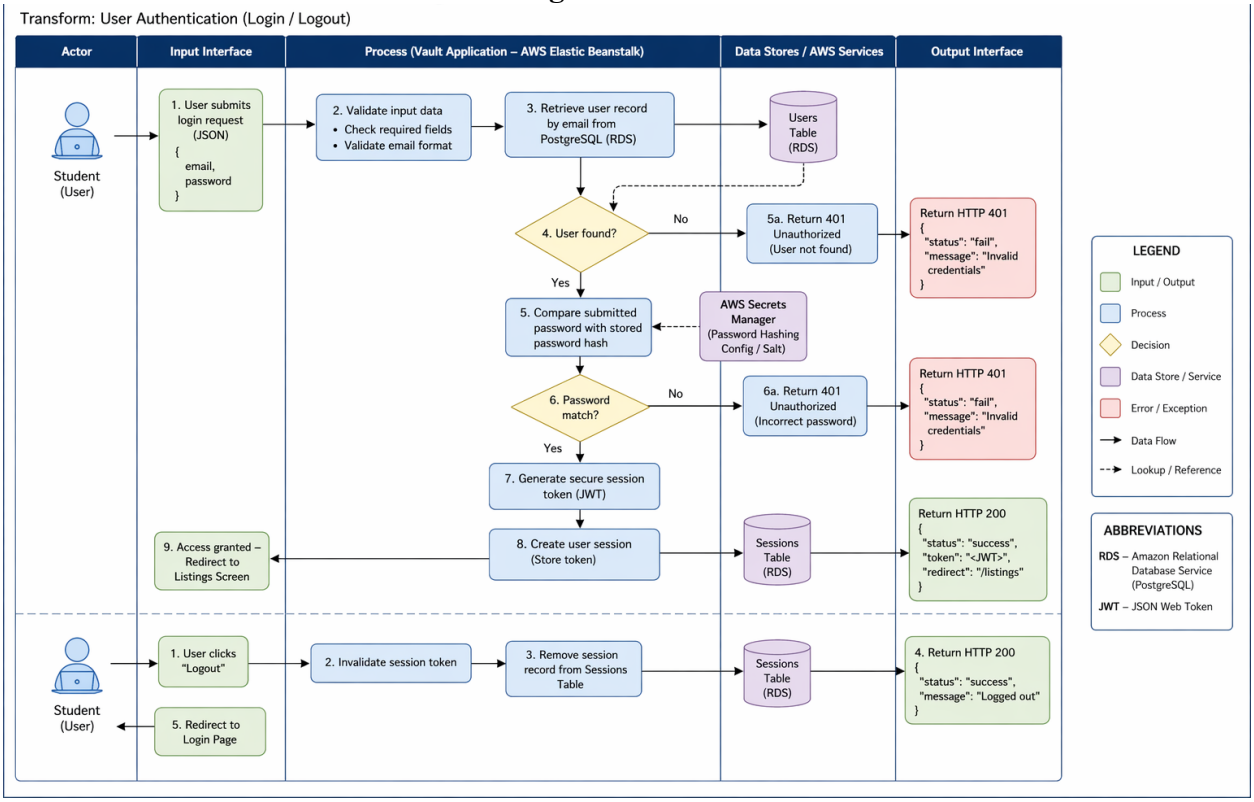
When logging in, the system retrieves the stored hash and compares it to the input password. If they match, a secure session token is generated to grant access to the site's features.

4.1.4.2 Transform 1 interface description

For the input interface, the system expects a JSON list containing the user's email, username, and password. If the data types are incorrect, the interface rejects the input before it even reaches the database.

For the output interface, a “200” success code is returned during a successful login and routes the user to the listing screen. A failed login sends a “401” error code.

4.1.4.3 Transform 1 lower level flow diagrams



4.1.4.4 Transform 2 description

This transform handles how brand owners control their products.

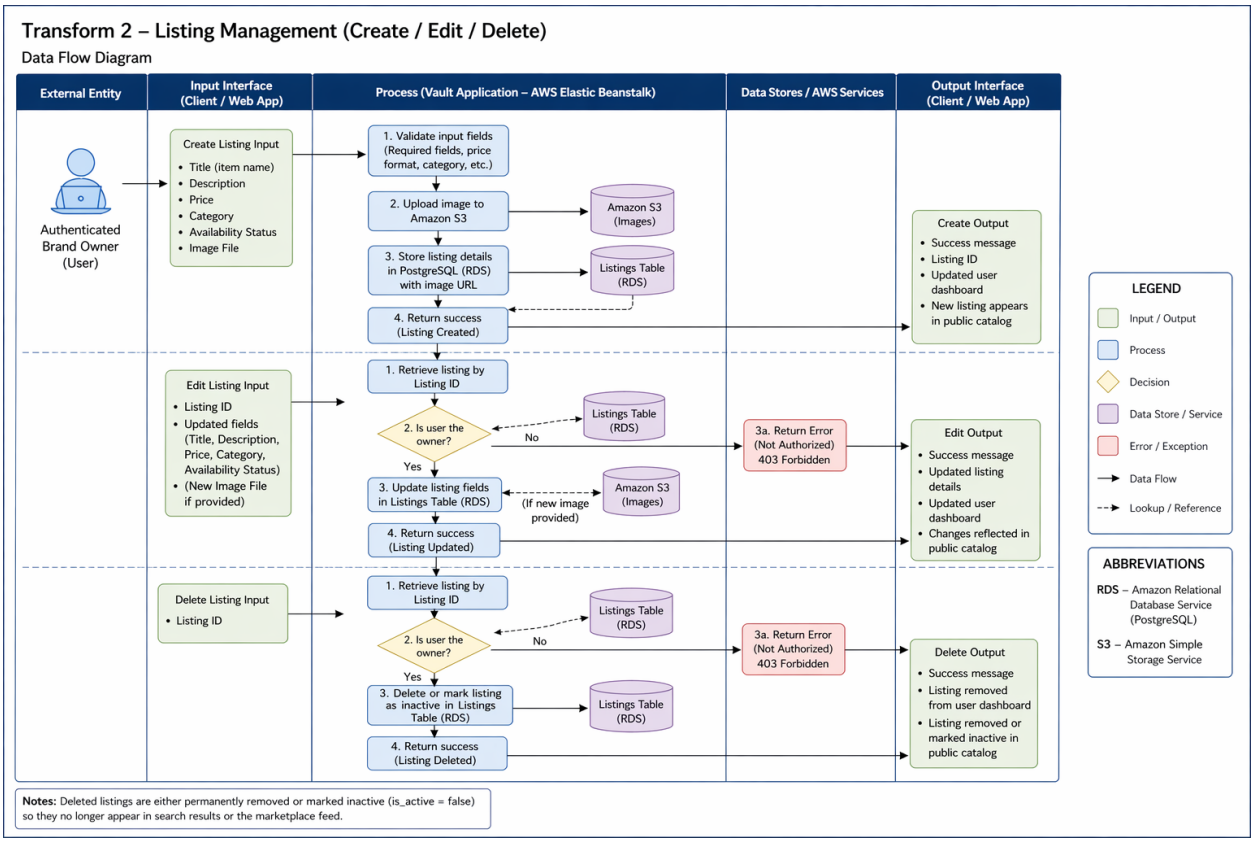
During listing creation, authenticated users input an item’s name, description, price, and category. The system validates the fields, stores the details in the RDS database, and uploads the product image to Amazon S3 storage.

When it comes to edits and deletion, the system verifies that the user is the actual owner of the listing before allowing any modifications or deletions. If deleted, the record is removed or marked inactive so it doesn’t appear in the feed anymore.

4.1.4.5 Transform 2 interface description

For the input interface, item details (title, price, availability status) are provided during listing creation. For the output interface, a public catalog of active listings and updated user dashboards showing their managed storefront is displayed.

4.1.4.6 Transform 2 lower-level flow diagrams



4.1.4.7 Transform 3 description

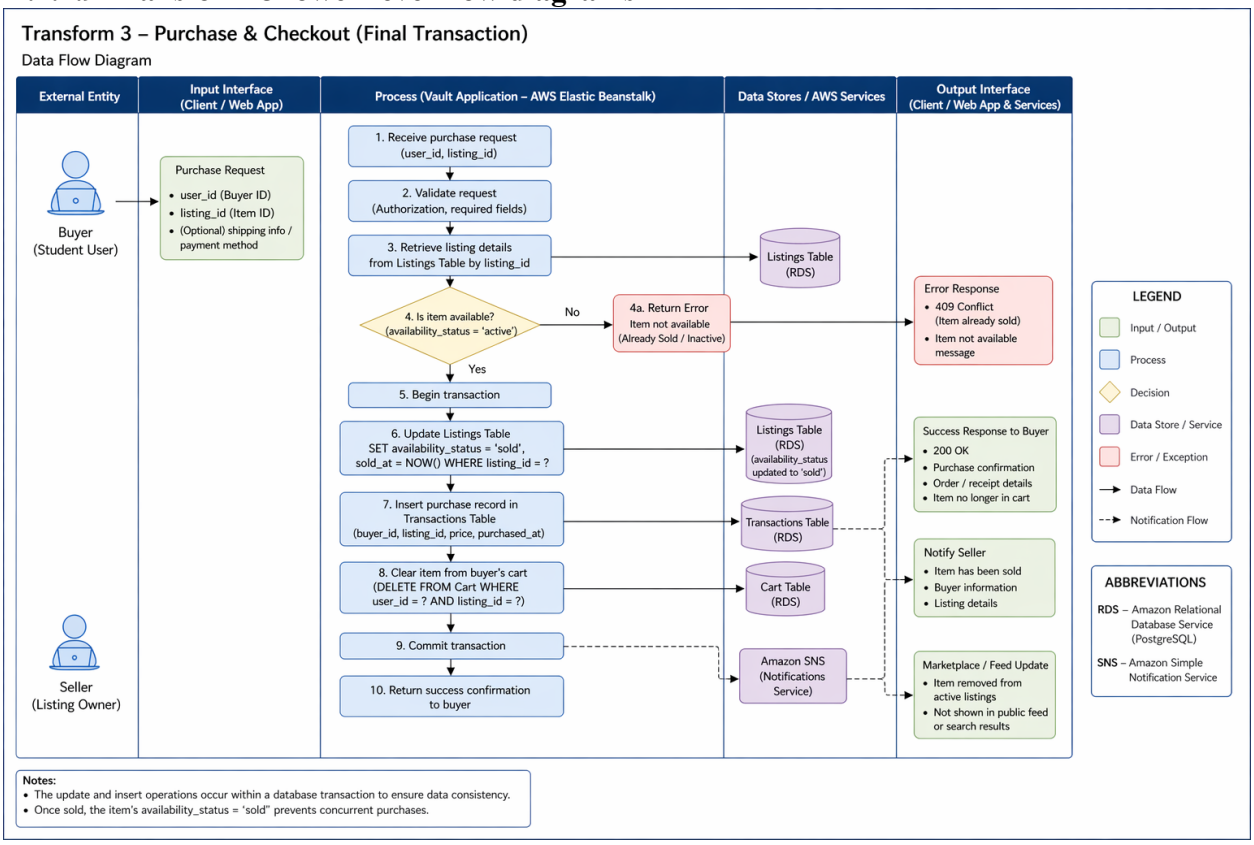
This transform handles the final transaction phase where a student buyer confirms their interest and purchases the item from the marketplace.

During purchase, the system queries the listings table to ensure the availability_status is still active to prevent concurrent purchases of a unique item. Once availability_status is verified, the backend executes a SQL UPDATE command to change the item's status to sold. The system then records the transaction details and links the buyer's user_id to the listing_id in a history table. Finally, the system clears the item from the user's cart.

4.1.4.8 Transform 3 interface description

The inputs are the User_ID of the buyer and the Listing_ID of the specific item being purchased, and the outputs are the updated "Sold" status in the database, a confirmation message to the buyer, and a notification sent to the seller. In addition, the item is removed from other users' feeds.

4.1.4.9 Transform 3 lower-level flow diagrams



4.1.5 Performance Issues

Special performance requirements for the subsystem are specified.

- Latency: Requests must travel from the user to the AWS RDS instance in us-east-1. The goal is to stay under 200ms query response times.
- Concurrency: The system should be able to handle multiple simultaneous user inputs without locking database rows.

4.1.6 Design Constraints

Any design constraints that will impact the subsystem are noted.

- Cloud security: The system is constrained by the AWS Security Group Inbound Rules. These only accept traffic from authorized IP addresses.
- Statelessness: Sessions are stored in secure cookies instead of files to allow scalability in Flask.

4.2 Software Interface Description

The software interface(s)to the outside world is(are) described.

4.2.1 External machine interfaces

Interfaces to other machines (computers or devices) are described.

- Psycopg2 communicates with AWS RDS over port 5432. This connection is initialized in init_db.py and tested in db.py.

4.2.2 External system interfaces

Interfaces to other systems, products, or networks are described.

- AWS RDS: The Vault uses AWS RDS as a global database. The backend uses the psycopg2 adapter to open a secure communication channel over the TCP/IP network on port 5432 to the AWS PostgreSQL instance. This connection is protected by an AWS Security Group that only allows communications from explicitly approved IP addresses.
- Internet Browser: The server interfaces with the browser using JSON for data and HTML/CSS for visuals.

4.2.3 Human interface

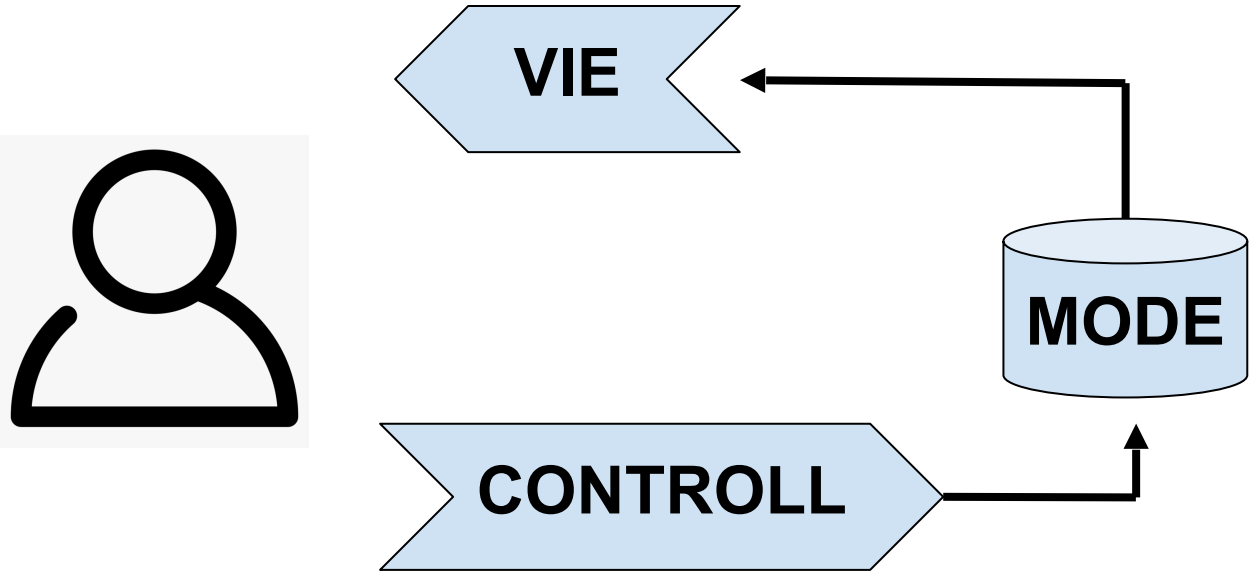
An overview of any human interfaces to be designed for the software is presented.

- Web GUI: A webpage at https://127.0.0.1:5000 will contain the webpage frontend.
- Admin CLI: init_db.py allows for direct database schema management.

4.3 Control flow description

The control flow for the system is presented with reference to Section 5.0 of this document.

1. Entry: Requests go to app.py.
2. Routing: app.py delegates to a routing blueprint.
3. Logic: The controllers validate the formatting of the requests and hand them to the services.
4. Data: The services request data from the models; the models execute the SQL commands on the AWS RDS.
5. Exit: Data flows back up the chain and is returned to the user as JSON.



4.4 Inherted Code

4.4.1 Code Contributions

Iteration 3 code contributions include three backend files added and one modified in order to support administrative functionality in the Vault project. These updates allow authorized administrators to monitor activity on the platform and remove inappropriate users or listings. The purpose of these changes is to implement a structured admin control system using the existing backend architecture.
Files created/added to:

- admin_controller.py
- admin_service.py
- admin_model.py
- app.py

admin_controller.py

This file defines the API endpoints that allow administrators to interact with the system. It creates a Flask Blueprint called admin_bp, which groups all admin-related routes under the /admin URL prefix. The controller receives HTTP requests from the frontend and calls the appropriate service functions to process the request. These routes allow administrators to retrieve all users, delete users, retrieve all listings, delete listings, and view storefronts. The controller also ensures that only authorized administrators can perform these actions before forwarding requests to the service layer. Once the request is processed, the controller returns the results to the frontend in JSON format.

admin_service.py
The service layer contains the business logic that supports administrative actions. It acts as the middle layer between the controller and the model. The service validates input parameters, enforces any rules for administrative operations, and calls the model functions that interact with the database. For example, when an admin requests to delete a user or listing, the service verifies that the required ID was provided before passing the request to the model. This separation helps maintain clean architecture by keeping validation and system rules out of the database layer.

admin_model.py
This file manages all database-level operations related to administrative functionality. It connects to the PostgreSQL database using the shared database connection defined in db.py. The model executes SQL queries to retrieve or modify data in the database. The functions in this file allow administrators to retrieve all users, retrieve all listings, retrieve storefronts, and permanently delete users or listings. The model layer focuses strictly on database interaction and does not contain any business logic.

app.py
The app.py file was modified to register the new admin routes with the Flask application. Specifically, the file imports the admin_bp Blueprint from controllers.admin_controller and registers it using app.register_blueprint(admin_bp). This step is necessary because Flask applications must explicitly register Blueprints before their routes become accessible. Without this change, the admin endpoints such as /admin/users or /admin/listings would not be recognized by the application. Registering the Blueprint integrates the admin functionality into the existing backend API structure alongside the listing and storefront routes.

Iteration 4:
Iteration 4 code contributions include three frontend files added and two modified in order to support administrative functionality in the Vault project. These updates created the frontend for the previous admin backend files created in the last iteration. Allow authorized administrators to monitor platform activity and remove inappropriate users or listings. The purpose of these changes was to implement a user-friendly UI that users with admin access can use to effectively complete admin activities.
Files created/added to:

- admin_dashboard.html
- admin.css
- admin.js
- auth_controller.py
- admin_controller.py

admin_dashbaord.html
This file defines the structure and layout of the admin dashboard interface. It creates a simple webpage with three main sections: Users, Listings, and Storefronts. Each section includes a button that lets the administrator load data from the backend, and a table that displays it dynamically. The file links to the external CSS file for styling and the JavaScript file for functionality. Overall, this file serves as the main user interface that administrators interact with after logging in.

admin.css
This file controls the visual styling of the admin dashboard. It defines the overall look and feel of the page, including the light gray background, white section containers, table formatting, and button styles. It also includes specific styling for delete buttons, making them red. The purpose of this file is to improve the readability and organization of the dashboard without affecting its functionality.

admin.js
This file handles the dynamic functionality of the admin dashboard by interacting with the backend API. It contains functions that send HTTP requests to retrieve users, listings, and storefronts, as well as functions to delete users and listings. After receiving data from the backend, the JavaScript dynamically updates the HTML tables to display the information. This file makes the dashboard interactive and ensures administrators can perform actions without reloading the page.

Iteration 5:
Iteration 5 code contributions included 2 completely new files, admin_login.html and test_admin.py. It also included updates or modifications made to 8 other code files with the purpose of improve both functionality and appearance of the admin dashboard. The purpose of these new files and updates were to incorporate a login page for admin users of The Vault, as well as expand on our previous version of the admin dashboard that was lacking in depth.
Files created/added to:

- test_admin.py
- admin_login.html
- Auth_controller.py
- admin_model.py
- admin_dashboard.html
- login.html

- admin.css
- admin.js
- auth_controller.py
- admin_controller.py

test_admin.py
This file contains both unit tests and integration tests for the admin dashboard functionality. The unit tests verify that routes, authentication, and API responses behave correctly by simulating requests and using controlled test data. The integration tests connect to the real PostgreSQL database and confirm that operations such as deleting users, listings, and storefronts actually modify the database as expected.

admin_login.html
This file defines the structure and layout of the administrator login page. It provides a dedicated interface separate from the standard user login, allowing administrators to securely authenticate before accessing the admin dashboard. The page includes input fields for username and password, styled consistently with the existing application theme to maintain a unified user experience.

4.4.2 Code Contributions

Elali was responsible for creating the backend portion of the wishlist and purchase history features for The Vault marketplace product. Due to the frontend portion of the features not being ready to be implemented yet, the code will have to remain unused and inoperational for now.
Files Created/Added to:

- Init_db.py
- app.py
- wishlist_service.py
- purchase_service.py
- wishlist_model.py
- purchase_model.py
- wishlist_controller.py
- purhcase_controller.py

The service, model, and controller files were created not only for connectivity with the corresponding data tables but also serves as a setup placement for the frontend portion of the code. The wishlist and purchase history features will be active in nearly every part of the marketplace, either due to UI design or data recall with the listing data structures. Since both

features will be created and regularly updated by the user they were added to the services folder and perform similar to the listing_service.py file. With the same reasoning and purpose, these features were added to the model and controller folders as well due to the constant potential editing or updates the user will make. The additions to the init_db.py were to serve as the initial data tables the user's entered information will be saved as, while the app.py additions were made to check the features functionality during the creation of the application.

Iteration 4

Elali was responsible for creating the front end portion of the "Create Listing" feature, "Wishlist" feature, and "Purchase History" feature for The Vault marketplace product. Now that the frontend portion of code has been created and implemented the backend portion of code for all three of these features is operational along with the frontend portion.

Files Created/Added to:

- app.py
- listing_controller.py
- Storefront_controller.py
- create_listing.py
- create_storefront.css
- storefront_view.css
- create_listing.js
- storefront_view.js
- account.html
- create_listing.html
- storefront.html
- storefront_view.html

App.py, listing_controller.py, storefront_controller.py, create_storefront.css, storefront_view.css, storefront_view.js, account.html, storefront.html, and storefront_view.html were all edited to route properly to my backend portion of code and the files I created. Create_listing.py, create_lisitng.js, and create_listing.html were created this Iteration to complete the frontend portion of these features. The "storefront" files, along with account.html and app.py, were edited to properly implement the "Wishlist" feature. The "listing" files, along with account.html, app.py, and listing_controller.py were edited if not created to properly implement the "Create Listing" feature.

Iteration 5

Elali was responsible for creating the test files for the "Create Listing" and "Wishlist" features. Along with the test files, Elali also made functional edits to both of these features as well.

Files Created/Added to:

- Listing_controller.py
- Requirements.txt
- Storefront_view.css
- Create_listing.js
- Storefront_view.js
- Create_listing.html
- Test_create_listing_feature.py
- Test_wishlist_feature.py

Listing_controller.py, Requirements.txt, Storefront_view.css, Create_listing.js, Storefront_view.js, and Create_listing.html were all edited to change the wishlist icon to make it more appealing and to make sure the listings being created or edited were saved and properly updated regularly. The new files created were test_create_listing_feature.py and test_wishlist_feature.py. They were test files created to check if both features were working properly and performing their tasks.

4.4.3 Code Contributions

Madison was responsible for the User Settings, implementing a modular design the manage user preferences and application configuration. She created the module using the MVC (model view controller) and service pattern to ensure separation and modularization of the application. She created a data model to define settings attribute, a controller to manage user driven requests, and a service layer to facilitate file I/O operations and persistence. Additionally, she implemented a testing file to validate data integrity, ensuring the application correctly handles both valid user inputs and invalid edge cases during file operations.

Files Created/Added to:

- usersettings.py the core service file responsible for reading, writing, and persisting user configuration data to the local file system, added usersettings.py file in models to define the structure and data attributes for the user settings object. Added a usersettings.py file in the controller to manage the interface between the user/UI and the settings logic, routing requests to the correct services. I added a usersettings.py in the services folder which summarizes the logic and file handling procedures for the settings module.
- usersettingstest.py testing file that validates the settings logic, including input validation and file integrity checks

Iteration 4

Madison was responsible for designing and implementing the Account Screen for The Vault front-end, focusing on creating an intuitive and user-friendly interface for managing user activity. She developed the account.html file to structure and display key user information, including listings, wishlist, purchases, and sales history. The interface was designed using a tab-based navigation system, allowing users to seamlessly switch between different categories of their account data.

She ensured the layout clearly presents the user's username at the top of the screen, showing personalization and user context. Additionally, she incorporated navigation functionality that allows users to easily return to the storefront or log out of their account.

Files Created/Added to:

- account.html – Core front-end file responsible for structuring the Account Screen UI, including tab-based navigation for user listings, wishlist, purchases, and sales history. Also includes display elements for the username and navigation options such as returning to the storefront and logging out.

Iteration 5:

Madison expanded her contributions by enhancing the testing and validation of the Settings functionality for *The Vault*, with a focus on ensuring reliability and integration with other account features. She updated the usersettingstest.py file to include additional unit and integration tests that verify both core settings behavior and its interaction with the broader application.

She implemented tests to confirm that account screen buttons displayed within the settings page correctly navigate users to their respective sections, ensuring consistency between the Settings interface and the Account Screen. This helped validate that the front-end navigation she previously developed functions properly across multiple components of the application.

In addition, she developed tests to verify that purchase history updates are accurately reflected within the user interface after transactions are completed. This ensures that user data remains synchronized between the settings, account screen, and backend systems.

Files Created/Added to:

- usersettingstest.py – Updated to include additional unit and integration tests for Settings functionality, including validation of account screen button navigation and purchase history updates.

4.4.4 Code Contributions

Ryan was responsible for the user registration and signup backend logic. He was able to create a file that connects the IDE (VS Code Studio) to the AWS RDS PostgresSQL instance, a file that initializes the user table with attributes (unique ID, username, email, password, role [admin or user]), a file that tests the IDE's connection to the database, a file that tests if users' credentials are being stored in the database, a file that tests the cart's functionality, a file that registers users and checks Hampton University association, a file that verifies a user's credentials during the login process by fetching the username from the database and uses an imported file that compares the password hashes, and a file that routes users between pages based on interactions and permissions. When it comes to user interface, Ryan designed the screens for the sign up and log in process in addition to the cart screen and backend.

Files Created/Added to:

- App.py: Executes the software.
- auth_utils.py: Hashes passwords and verifies hash comparisons.
- auth_services: Register user and validate login.
- auth_controller.py: Routes user based on permissions.
- db.py: Connects IDE to AWS RDS.
- init_db.py: Creates table schemas.
- Index.html: Front-end for welcome screen
- Signup.html: Front-end for sign up screen
- Login.html: Front-end for login screen
- Cart.html: Front-end for cart screen
- test_user_driver.py: Tests ability to store and retrieve credentials
- test_db_driver.py: Tests database connection
- test_cart_functionality_driver.py: Tests cart functions

4.4.5 Code Contributions

Iteration 3 Code contributions include files added to the backend development of The Vault by implementing the core components responsible for storefront and listing management within the system.

Files Created/ added to:

- Listing_model.py

- Listing_controller.py
- Listing_service.py
- Storefront_model.py
- Storefront_service.py
- Storefront_controller.py
- db.py
- Init_db.py
- mock_data.py

Focused on enabling users to create storefronts, publish listings for their products, and manage inventory while making sure everything works correctly with the database.
More specifically, the development of the storefront management functions (storefront_models.py, storefront.services.py, and storefront_controller.py). These files allow a registered user to create and maintain a storefront associated with their account. The storefront stores information such as the brand name, description, and any visuals such as logos or banners. Regarding the database structure, I designed it in a way that each storefront is tied to a specific user account through a foreign key relationship which ensures proper ownership and prevents unauthorized access.
The development of the listing management functionality (listing_model.py, listing_service.py, listing_controller.py) allows storefront owners (which are now considered brand owners) to create and manage listings within their storefront. Listings contain attributes such as title, description, price, and inventory information. It was designed for the system to support both inventory-based listings and pre-order listings, allowing brand owners to either manage existing stock or accept orders that are produced after purchase. Additional listing features implemented were listing image functionality and size based inventory tracking. The system allows multiple images to be associated with each listing and the system stores image URLs that reference files stored in S3. The system also now allows for clothing items in each listing to have different stock quantities depending on size, so each listing can track available quantities for individual sizes and it also auto updates to SOLD_OUT once quantity reaches 0. This makes the inventory system a lot more realistic.
Contributions to the database schema design, with creating several key tables including:

- Storefronts
- Listings
- Listing_images
- listing_sizes

Which were all implemented in the init_db.py file as well.
Lastly, there were additional files such as db.py, and mock_data.py added in order to successfully connect our new code structure to the database and test it.

4.4.5.1Coding Contributions - I4

Iteration 4 code contributions include files added to the frontend development of The Vault, implementing the core components responsible for the storefront browsing and creation experience.

Files Created/Added to:

- Storefront.html
- Storefront.css
- storefront.js
- Storefront_view.html
- Storefront_view.css
- Storefront_view.js
- Create_storefront.html
- Create_storefront.css
- Create_storefront.js
- Storefront_controller.py (updated)

storefront.html Serves as the main skeleton and entry point for the storefront homepage. Defines the page structure including the top navigation bar with bag and profile icon routing, a side navigation pane with Create Storefront and Create Listing buttons, a logo and search section, and the storefront card grid container where cards are injected dynamically by JavaScript. The pagination controls at the bottom of the page are also defined here.

storefront.css Handles all styling and layout for the storefront homepage. Styles the top navigation bar, menu button, side navigation pane, brand title, search bar, storefront cards, listing preview images, Add to Bag and Enter Storefront buttons, and pagination controls. Includes responsive design considerations to maintain consistent layout across different screen sizes.

storefront.js Dynamically generates storefront cards by fetching data from the backend API. Renders each card with the store logo, brand name, listing previews, and an Enter Storefront button that routes to the individual storefront view.

storefront_view.html Defines the page structure for the individual public-facing storefront page. Includes the banner, overlapping logo, store description section, and listing grid container.

storefront_view.css Controls the styling and layout of the storefront view page. Styles the banner, overlapping logo, back button, brand info, store description, and listing grid.

storefront_view.js Fetches and renders individual storefront data dynamically using the storefront ID from the URL. Populates the banner, logo, description, and listing grid with live data from the API.

create_storefront.html Defines the page structure for the storefront creation form. Includes input fields for brand name, description, logo, banner, contact info, categories, and preview images.

create_storefront.css Handles all styling for the create storefront form page. Styles the form fields, upload sections, category checkboxes, and action buttons.

create_storefront.js Captures form input on submission and sends it to the backend to create a new storefront. On success, the new storefront is reflected as a card on the homepage.

storefront_controller.py (updated) Updated to add Flask page routing for the storefront homepage, storefront view, and create storefront pages. Routes are registered under the storefront_pages_bp blueprint.

4.4.5.2 Coding Contributions - I5

Iteration 5 code contributions include files added and updated to complete the storefront system, S3 image upload integration, listing lifecycle management, returns system, and database schema migrations for The Vault.

Files Created/Updated:

- utils/s3.py
- storefront_controller.py (updated)
- storefront_service.py (updated)
- storefront_model.py (updated)
- storefront.html (updated)
- storefront.css (updated)
- storefront.js (updated)
- storefront_view.html (updated)
- storefront_view.css (updated)
- storefront_view.js (updated)
- my_storefront.html
- my_storefront.css
- my_storefront.js

- create_storefront.html (updated)
- create_storefront.css (updated)
- create_storefront.js (updated)
- edit_storefront.html
- edit_storefront.js
- listing_controller.py (updated)
- return_controller.py
- return_service.py
- return_model.py
- returns.html
- returns.js
- init_db.py (updated

utils/s3.py

Created to handle all image upload functionality for the application using boto3. Provides the upload_image_to_s3() function which uploads image files to the configured S3 bucket and returns a public URL, used across storefront creation, listing creation, and return submissions.

storefront_controller.py

Updated to add the S3 image upload endpoint, page routing for the edit storefront page, edit listing page, and my storefront dashboard. Also added the storefront reactivation endpoint to restore a previously deactivated storefront and its associated listings.

storefront_service.py

Updated to add reactivate_storefront_service(), normalize_storefront_categories(), and support for the new storefront fields including contact info, preview images, and categories in both the create and update service functions.

storefront_model.py

Major refactor to support the expanded storefront schema, updating all model functions to include the new fields added to the storefronts table and fixing get_active_storefronts() to correctly filter by active status.

storefront.html / storefront.css / storefront.js

Updated to pull live storefront data from the database via the backend API, replacing the previous static implementation. Added real-time search functionality to filter storefronts based on user input.

storefront_view.html / storefront_view.css / storefront_view.js

Updated to dynamically fetch and render storefront data using the storefront ID from the URL, including banner, logo, store description, contact info, and listing grid. Added a lightbox image viewer for listing image previews.

my_storefront.html / my_storefront.css / my_storefront.js

New files created to implement the My Storefront dashboard page where brand owners can manage their storefront and listings. Includes buttons to edit, create listings, view the public storefront, and deactivate.

create_storefront.html / create_storefront.css / create_storefront.js

Updated to support the expanded storefront creation form including contact info, categories, and up to four preview images. The JavaScript was updated to handle S3 image uploads before form submission.

edit_storefront.html / edit_storefront.js

New files created to implement the edit storefront page, pre-populating all existing storefront fields on load and allowing the brand owner to update any field including uploading new images to S3.

listing_controller.py

Updated to integrate S3 image uploads into the listing creation flow supporting up to four images per listing. Added deactivate, reactivate, and restore endpoints for listing lifecycle management.

return_controller.py / return_service.py / return_model.py

Created to implement the full returns system backend, handling return form submissions including optional damage image uploads to S3, service-layer validation, and database record creation.

returns.html / returns.js

New files created to implement the returns page frontend where users can submit a return request with order number, reason, damage status, and an optional damage image upload.

init_db.py

Updated to add the orders, order_items, and cart_items table schemas and ALTER TABLE migration statements for new columns added to the storefronts and listings tables, executed on every application startup.

4.4.6 Code Contributions

For iteration 3, my code contributions consist of constructing and adding to the following files: listings.py, account_controller.py, and accountscreen.py.

listing.py

- This file is a part of our front-end development. Its purpose is to create a browser listing screen that will hold all listings created that are readily purchasable or available for pre-order. It allows users to view all listings in addition to our additional screen, which will be used to allow customers to view personal stores. Just like that, the screen, once a listing is chosen, will create a pop-up box and allow the user to view the listing's information. Creating this file, I used Python Tkinter for functionality. It also uses the requests library to communicate with the backend controller (listing_controller.py) and fetches the JSON data to build the marketplace grid.

Account_controller.py

- This file is a part of our back-end development. It supports the student dashboard or account screen, which will manage all user-specific data transactions. This file was made to process requests that involve student personal information and utilizes SQL queries that have parameters to fetch data based on the user_id for each user. In this document I was able to implement multi-table inquiry logic that will assist in verifying if the specific user has a storefront and any listings. Upon further testing, I will ensure the account management system will be able to be scaled independently as more features are added.

AccountScreen.py

- This file is a part of our front-end development. It serves as a centralized dashboard for managing account presence and more within the application. Using the Tkinter library, I created a modular layout that organizes user data into sections while using LabelFrame widgets. In this screen, users will be able to view five primary attributes: name, verified student email address, saved payment methods, purchase history, and (if applicable) personal listings. The interface is to be responsive and would update its view once navigated to, which ensures users are viewing their most recent/current account status.

Iteration 4 Contributions

- For iteration 4, my code contributions consist of constructing and adding to the following files: **checkout.html**, **checkout.css**, **checkout.js**, and **app.py**.

checkout.html This file is a primary component of our front-end web development. Its purpose is to provide a dedicated "Finalize Purchase" interface where users can complete their transactions after browsing the marketplace. I structured this page to include a dual-column layout: one section for collecting essential buyer data—such as full name, campus email, and physical meeting location—and a sidebar for the order summary. I implemented Flask's Jinja2 templating to include dynamic navigation, specifically adding a "Back to Cart" function that allows users to return to their bag without losing their session data.

checkout.css The checkout experience's visual architecture is managed by this file. I used CSS Flexbox and custom linear gradients to match the storefront's gold-on-black theme in order to preserve brand coherence with "The Vault" aesthetic. I used metallic borders and semi-transparent backgrounds to create particular "glass-morphism" effects for the form containers. To make the interface feel responsive and high-end for the user, I also created unique hover states and transitions for the "Complete Transaction" and "Back to Cart" buttons.

Checkout.js The client-side logic needed for a working marketplace is handled by this file. It acts as a link between the user interface and the browser's storage. In order to dynamically create the list of purchased items and determine the final order total, I wrote scripts to retrieve JSON data from localStorage. In order to manage form submission, which clears the user's cart upon a successful transaction and initiates a confirmation alert, I also implemented event listeners. This guarantees that the purchase process is interactive and gives the student buyer instant feedback.

app.py My contributions to this back-end file involved managing the routing and integration of the checkout module within the Flask framework. I defined the @app.route('/checkout') function to render the checkout template and ensured all static assets were correctly linked. I worked on the integration logic to ensure that navigation between the Cart, Listing Screen, and Checkout remained seamless. By managing merge conflicts on the development branch, I successfully integrated my new web routes with the team's existing codebase, ensuring a stable environment for further testing.

Iteration 5 Contribution -

For this iteration, my code contributions consist of constructing, refining, and integrating multiple components related to the checkout system, including backend logic, controller functionality, testing, and post-transaction features. I contributed with the following files: checkout_models.py, checkout_service.py, checkout_controller.py, order_confirmation.html, and test_checkout.py.

checkout_models.py
This file serves as the data access layer for the checkout system. Its purpose is to handle direct communication with the database and retrieve all necessary information required during the checkout process. I structured this file to support retrieving user information, cart contents, and item pricing. This includes pulling stored user data for existing accounts so their information can be automatically populated during checkout, as well as retrieving all items currently in the cart.

checkout_service.py
This file contains the business logic that drives the checkout process. I corrected and implemented the functionality responsible for calculating the total price of items in the cart, ensuring accurate accumulation of item prices and quantities. Additionally, I developed logic to manage the checkout workflow, including processing cart data, validating user input, and preparing order details for completion. Lastly, I also implemented functionality that clears the user's cart once a purchase is successfully completed.

checkout_controller.py
This file is responsible for handling the flow of data between the frontend and backend during the checkout process. I contributed to structuring controller logic that connects user actions, such as initiating checkout and completing a transaction, with the appropriate service functions. This includes routing users through the checkout process and ensuring that once a transaction is completed, they are redirected to the order confirmation page. This file plays a key role in maintaining a smooth and functional user experience.

order_confirmation.html
This file provides the user interface displayed after a successful transaction. Its purpose is to confirm that an order has been processed and to provide the user with clear feedback. I contributed to designing this page to display confirmation messaging and relevant order details, ensuring consistency with the overall checkout design. This page completes the transaction flow by giving users reassurance that their purchase was successful.

test_checkout.py
This file was created to support testing and validation of the checkout system. Its purpose is to ensure that all major checkout functionalities perform correctly. I designed and implemented test cases to verify total cost calculations, validate correct retrieval of user and cart data, and ensure that the checkout process executes without errors. This establishes a foundation for automated testing and improves overall system reliability.

5.0 Behavioral Model and Description

A description of the behavior of the software is presented.

5.1 Description for software behavior

This section describes the dynamic behavior of The Vault system, including the major system states and the events that trigger transitions between those states. The behavioral model illustrates how the system responds to user interactions, administrative actions, and internal validation processes.

The Vault is currently operating as a stateless RESTful API, where transitions are governed by session-based authentication and Role-Based Access Control (RBAC). The default state is unauthenticated. In this state, they only have access to the registration/login endpoints. Once a user is validated with a valid Hampton email and password match, they enter an authenticated state. In this state, they have create, read, edit, and delete privileges for listings. Admins have a privileged authenticated state that grants them global read, write, and delete privileges.

5.1.1 Events

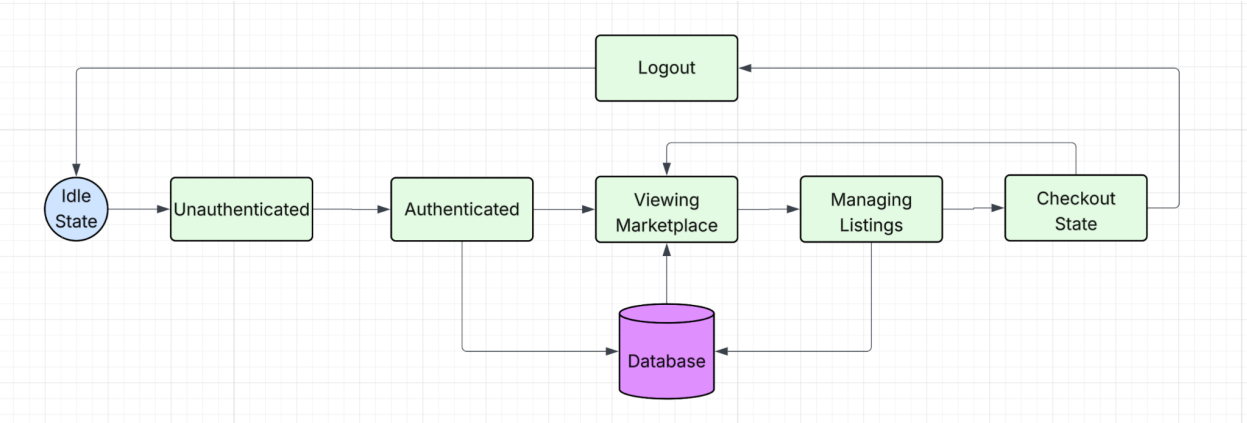
Event	Description
Register Submitted	User provides information to register
Login Submitted	User tries logging in
Create listing clicked	User begins creating their listing
Submit listing	User submits their listing
Edit listing clicked	User edits their listing
Delete listing clicked	User removes their listing
Search performed	User searches for specific listings
Purchase confirmed	User confirms purchase
Admin remove	Admin removes a listing
Logout	User ends session and leaves the webpage

5.1.2 States

The system operates under the following primary states:

1. Unauthenticated State
2. Authenticated State
3. Viewing Marketplace
4. Creating Listing
5. Editing Listing
6. Checkout State
7. Admin State

5.2 State Transition Diagrams



5.3 Control specification (CSPEC)

The system uses event-driven control logic:

- User input triggers events.
- Backend validates state.
- If valid, state transition occurs.
- If invalid, error state is returned.

Security control rules:

- Only Hampton University students can use *The Vault*, security verification th
- Only authenticated users may create/edit/delete listings.
- Only admin users may remove content.
- Session timeout returns system to Unauthenticated state.

6.0 Restrictions, Limitations, and Constraints

Restrictions

- **Hampton University-Only Access:** The Vault is restricted to use by Hampton University students only. A Hampton University email address is required to sign in and/or create a

new account. The system will validate if it is a Hampton University email account based on the ending of the email address being (hamptonu.edu).

- **Role-Based Access Control:** Users are restricted based on assigned roles. Brand owners and shoppers cannot access administrative controls and only administrators can manage all users and listings. This restriction enforces security and ownership boundaries in the system.
- **AWS Free Tier Deployment Requirement:** The system must be deployed using AWS Free Tier services. This restricted available computing resources, database sizes and storage limits, and performance abilities.

Limitations

- **AWS RDS PostgreSQL Query Editing Limitation:** When Postgres is implemented through RDS AWS the free tier does not allow for the query to be edited directly through AWS. To circumvent this, we made an init_db.py in VS Code that can initialize the tables in Postgres without having to upgrade the AWS plan
- **Scalability Limitation Under Free Tier:** Because the project has to be operated within the AWS Free Tier, the database storage, memory allocation, and connections are limited. Performance under high traffic or large numbers of users may be limited.
- **Development Timeline Limitation:** The project is limited by the semester timeline. Some advanced features may not be fully implemented before the project is completed.

Constraints

- **Technical Constraints:** The system must be compatible with AWS Hosting services, specifically AWS RDS for database management. The application must support cross-platform browser compatibility/access like macOS and Windows. PostgreSQL is the required relational database system for efficient storage.
- **Security Constraints:** Passwords must be stored using secure hashing methods. Sensitive payment information cannot be stored in the database system. Listing ownership must be enforced at database and application levels.
- **Resource Constraints:** The team must work within the AWS Free tier system. Development must also be completed within the project iteration timelines.

7.0 Validation Criteria

Login Page

- Users are able to create an account
- Users are able to login to their already existing account
- The account login information will be stored into the database
- Once the login process is complete, the user will be brought to the listing page

Listings Page

- Users can view several different listing posted
- Users can create their own listing

- Users can edit their existing listing
- Users can select to buy a listing
- All user-created listing information will be stored into the database
- After the user selects to buy a listing, they will be brought to the checkout section

Checkout Page

- Users can confirm their purchase and submit their payment

7.1 Classes of tests

The types of tests to be conducted are specified, including as much detail as is possible at this stage. Emphasis here is on black- box testing.

- **Unit Testing:** This is when a tester tests small chunks of code, like a single function or method, to make sure it does what it's supposed to. The goal is to confirm that the logic the developer wrote works the way they intended (GeeksforGeeks, 2025). This will be easily implemented by the Vault because programmers will test functionality before changes are committed to the shared repository.
- **Integration Testing:** This type of testing checks how different pieces of the software work together. It makes sure that data flows correctly between components and that nothing breaks when modules are combined (GeeksforGeeks, 2025). This will consistently be implemented individually and collaboratively by validating changes before commitment and validating changes on each individual's desktop after commitment.
- **System Testing:** Once everything is integrated, system testing looks at the entire application as a whole. It checks if the full system meets the requirements and works the way it's supposed to (GeeksforGeeks, 2025). This won't be as heavily desired since a finished, working product isn't expected by the customer.
- **Black-Box Testing:** In black-box testing, the tester doesn't need to know how the system is built on the inside. They just provide inputs and check the outputs. This helps reveal how the system reacts to different user actions, whether expected or not, and can point out issues with usability, reliability, or performance. Common types include functional, non-functional, and regression testing (GeeksforGeeks, 2025;Imperva, n.d.). This may be difficult to implement because everyone is a programmer, so we're expected to know how the system is built on the inside. To mitigate this issue, we can have individuals who did not code the tested functions test the functions.
- **White-Box Testing:** In white-box testing the tester has full access to the code, structure, and configuration. It checks the logic, data flow, and error handling to make sure everything inside the program runs the way it should (GeeksforGeeks, 2025; Imperva, 2025). This will be heavily and consistently used throughout coding. White-box testing should be done by every member of the team, and the software lead should do a very in-depth white-box testing.

7.2 Expected software response

- **Scalability:** Ability to process high volumes of listings and users.
- **Security Compliance:** No unauthorized access to restricted data or features, encrypted storage of sensitive data.
- **Usability:** Clear and accessible interface, ensuring seamless user interactions.
- **Fault Tolerance:** The system must be able to remain stable during unexpected failures, with automatic recovery and fallback mechanisms.

7.3 Performance bounds

- **Admin Security:** Only authorized users should be able to access that admin page.

8.0 Revision Logs

Revision	By	Date	Description
1.0	The Vault	1/13/26-1/22/26	The initial document was created, and an outline and table of contents were added.
2.0	The Vault	1/23/26-2/12/26	Updated majority of the previously unfilled selections (2, 3, & 7)
3.0	The Vault	2/12/2026- 3/5/26	Updated Section 6 based on additional restrictions for the project, updated sections 4-5 with additional diagrams and functions present in our code, added a section 4.4 to include each team member's code contributions and descriptions

4.0	The Vault	3/5/26-3/26/26	Added code contributions for iteration 4
5.0	The Vault	4/9/26 - 4/23/2026	Added I5 code contributions for all team members (Sections 4.4.1-4.4.6). Updated data model sections 3.1.1.7-3.1.1.10 to reflect actual Order, Order Item, Cart Item, and Return objects. Updated 3.1.2 relationships, 3.1.3 complete data model, and 3.1.4 data dictionary to remove payment references and add cart, order, and return entities. Updated 3.0 and 3.1 intro to reflect completed checkout and order tracking system.